



LI.FI

LI.FI Security Review

SymbiosisFacet.sol (v2.0.0)

Security Researcher

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

July 21, 2026

Contents

1	About Researcher	2
2	Disclaimer	2
3	Scope	2
4	Risk classification	2
4.1	Impact	2
4.2	Likelihood	3
4.3	Action required for severity levels	3
5	Executive Summary	3
6	Findings	4
6.1	Medium Risk	4
6.1.1	Swap leftovers and excess native are refunded to msg.sender, stranding funds at any contract that calls the diamond on a user's behalf	4
6.2	Low Risk	4
6.2.1	Mutable OnchainSwapV3 fee can disable ERC-20 routes	4
6.2.2	OnchainSwapV3 signature verified against post-swap amount	5
6.3	Informational	5
6.3.1	Unvalidated approvedTokens[0] can steal Diamond balance	5
6.3.2	Swap entrypoint omits doesNotContainDestinationCalls	5
6.3.3	Source-swap output asset is not constrained to bridgeData.sendingAssetId	6
6.3.4	The documented intermediateToken field is never read, so it has no effect on routing	6
6.3.5	The MetaRouter branch accepts unauthenticated routing calldata emitting inaccurate LiFi-TransferStarted event	7

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- src/Facets/SymbiosisFacet.sol(v2.0.0)
- src/Interfaces/IOnchainSwapV3.sol(v1.0.0)

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

- High** leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium** global losses <10% or losses to only a subset of users, but still unacceptable.
- Low** losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

- High** almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium** only conditionally possible or incentivized, but still relatively likely
- Low** requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

- Critical** Must fix as soon as possible (if already deployed)
- High** Must fix (before deployment if not already deployed)
- Medium** Should fix
- Low** Could fix

5 Executive Summary

Over the course of 8 hours in total, LI.FI engaged with the [researcher](#) to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 8 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/contracts
Commit	74ade3e7
Audit Timeline	July 20, 2026 - July 21, 2026
Methods	Manual Review
Documentation	High
Test Coverage	Medium-High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	2
Gas Optimizations	0
Informational	5
Total Issues	8

6 Findings

6.1 Medium Risk

6.1.1 Swap leftovers and excess native are refunded to msg.sender, stranding funds at any contract that calls the diamond on a user's behalf

Context: [SymbiosisFacet.sol#L186-L206](#), [SymbiosisFacet.sol#L162-L180](#)

Description: Both entrypoints hard-code msg.sender as the refund destination for excess native, and for Swap-perV2's leftover receiver.

```
refundExcessNative(payable(msg.sender))
containsSourceSwaps(_bridgeData)
validateBridgeData(_bridgeData)
{
    _bridgeData.minAmount = _depositAndSwap(
        _bridgeData.transactionId,
        _bridgeData.minAmount,
        _swapData,
        payable(msg.sender)
    );
};
```

msg.sender is the fund owner only for a direct EOA call.

LI.FI's own Permit2Proxy invokes the diamond via LIFI_DIAMOND.call{value: msg.value}(...), so on every swap through it the leftover input token and any unspent msg.value land at the shared proxy, never the permit signer.

Recommendation: Add a refundRecipient field to SymbiosisData, revert InvalidCallData() if it is zero at the top of **every** entrypoint (before any deposit or swap), and pass it to both refundExcessNative and _depositAndSwap instead of msg.sender.

```
refundExcessNative(payable(_symbiosisData.refundRecipient))
// ...
if (_symbiosisData.refundRecipient == address(0)) revert InvalidCallData();
```

Do not reuse _bridgeData.receiver (it is the NON_EVM_ADDRESS sentinel on the OnchainSwapV3 path), and bind refundRecipient into SYMBIOSIS_PAYLOAD_TYPEHASH so a relay cannot substitute it on signed quotes.

LI.FI: Fixed in [14d32a4](#)

Researcher: Verified fix.

6.2 Low Risk

6.2.1 Mutable OnchainSwapV3 fee can disable ERC-20 routes

Context: Global

Description: The owner-controlled router fee is currently zero but can be changed through changeFee in the Symbiosis OnchainSwapV3 contract.

The facet forwards no native value for ERC-20 inputs, so any nonzero fee causes all ERC-20 OnchainSwapV3 routes to revert. For native inputs, the fee is deducted from _bridgeData.minAmount, potentially invalidating the signed calldata.

Recommendation: Read and account for the router fee in the signed quote and forwarded value, or explicitly enforce and monitor a permanent zero-fee requirement.

LI.FI: Fixed in [d6ccd2d](#)

Researcher: Verified fix. With the current fix, `swapAndStartBridgeTokensViaSymbiosis` requires the router fee to be supplied through `msg.value`; funding it through an ERC-20→native source swap is unsupported. This does not affect expected routes because the backend includes the signed router fee in the transaction's native value.

6.2.2 OnchainSwapV3 signature verified against post-swap amount

Context: [SymbiosisFacet.sol#L198-L205](#)

Description: `swapAndStartBridgeTokensViaSymbiosis` overwrites `_bridgeData.minAmount` with the exact amount `_depositAndSwap` measured at runtime, and only afterwards does `_startBridge` reach `_verifyOnchainSwapV3Signature`, which hashes that mutated value.

```
bridgeData.minAmount = 100_000_000; // signed, slippage-adjusted USDC floor; DAI->USDC returns  
↪ 100_450_000  
symbiosisData.viaOnchainSwapV3 = true;  
_bridgeData.minAmount = _depositAndSwap(...); // returns and assigns 100_450_000  
_startBridge(_bridgeData, _symbiosisData); // hashes 100_450_000, not the signed 100_000_000
```

The backend must sign before execution, so it can only ever sign the amount it quoted. On the swap entrypoint that amount is `bridgeData.minAmount` in its pre-swap meaning: the slippage-adjusted *minimum* acceptable output, not the expected output. `_depositAndSwap` (`SwapperV2.sol:119-126`) reverts only when `newBalance < _minAmount` and otherwise returns the realized balance, which for any ordinary route exceeds the slippage-adjusted floor. `ECDsa.recover` then yields an address other than `backendSigner` and the call reverts with `InvalidSignature`; the combination succeeds only when the DEX returns the signed amount to the wei, so the mismatch is the common case, not an occasional one.

Recommendation: Verify the signed quote against the unmodified pre-swap amount, before any deposit or swap, and treat the signed amount as the exact amount handed to `onswap`; refund the surplus of the realized output over the signed amount, and remove the post-swap verification inside `_startBridgeViaOnchainSwapV3` so the signature is checked exactly once.

```
if (_symbiosisData.viaOnchainSwapV3) _verifyOnchainSwapV3Signature(_bridgeData, _symbiosisData);  
uint256 received = _depositAndSwap(...); // leave minAmount at the signed amount; refund received -  
↪ signed
```

LI.FI: Fixed in [796f16d86](#)

Researcher: Verified fix.

6.3 Informational

6.3.1 Unvalidated `approvedTokens[0]` can steal Diamond balance

Context: [SymbiosisFacet.sol#L248](#)

Description: The facet forwards caller-controlled `approvedTokens` to the `MetaRouter` after granting the Symbiosis gateway persistent maximum allowances. The `MetaRouter` pulls `approvedTokens[0]` from the Diamond without verifying that it equals `_bridgeData.sendingAssetId`. A caller could therefore exploit this to steal any funds in the diamond.

Recommendation: Require `_symbiosisData.approvedTokens[0] == _bridgeData.sendingAssetId`, or use exact temporary allowances that are revoked after `metaRoute`.

LI.FI: Fixed in [30eb216](#)

Researcher: Verified fix.

6.3.2 Swap entrypoint omits `doesNotContainDestinationCalls`

Context: [SymbiosisFacet.sol#L196](#)

Description: `swapAndStartBridgeTokensViaSymbiosis` omits the `doesNotContainDestinationCalls(_bridgeData)` guard that the otherwise equivalent `startBridgeTokensViaSymbiosis` applies, and neither `validateBridgeData` nor the backend `SYMBIOSIS_PAYLOAD_TYPEHASH` signature ever inspects `hasDestinationCall`.

A call with `hasSourceSwaps = true`, `hasDestinationCall = true`, `receiver = NON_EVM_ADDRESS`, `destinationChainId = LIFI_CHAIN_ID_BTC` and `viaOnchainSwapV3 = true` therefore passes every modifier, burns syBTC to release BTC where no destination-side execution can occur, and emits `LiFiTransferStarted` with the false flag.

```
containsSourceSwaps(_bridgeData)
validateBridgeData(_bridgeData)
```

Recommendation: Add `doesNotContainDestinationCalls(_bridgeData)` to the swap entrypoint's modifier list, before `_depositAndSwap` moves any tokens — or, if `MetaRouter callTo/callData` is meant to be a real destination call, revert `InformationMismatch` on `_bridgeData.hasDestinationCall` inside `_startBridgeViaOnchainSwapV3` only.

LI.FI: Fixed in [4f05763](#)

Researcher: Verified fix.

6.3.3 Source-swap output asset is not constrained to `bridgeData.sendingAssetId`

Context: [SymbiosisFacet.sol#L198](#)

Description: `_depositAndSwap` measures its return value against the last swap's `receivingAssetId`, but both bridge branches then approve, spend and forward `_bridgeData.sendingAssetId`, and neither `containsSourceSwaps` nor `validateBridgeData` inspects `_swapData` assets.

```
_bridgeData.minAmount = _depositAndSwap(
    _bridgeData.transactionId,
    _bridgeData.minAmount,
    _swapData,
    payable(msg.sender)
);
```

Recommendation: At the top of `swapAndStartBridgeTokensViaSymbiosis`, before any deposit or swap, revert with `InformationMismatch` when `_swapData.length != 0 && _swapData[_swapData.length - 1].receivingAssetId != _bridgeData.sendingAssetId`, leaving the empty-array case to `_depositAndSwap` so its `NoSwapDataProvided` error is preserved.

LI.FI: Fixed in [7facf56](#)

Researcher: Verified fix.

6.3.4 The documented `intermediateToken` field is never read, so it has no effect on routing

Context: [SymbiosisFacet.sol#L103](#)

Description: Neither `_startBridgeViaMetaRouter` nor `_startBridgeViaOnchainSwapV3` reads `SymbiosisData.intermediateToken`; the `MetaRouter` payload is built from the surrounding fields only, and the upstream `MetaRouteTransaction` struct has no member that could carry it.

Two calls differing only in `intermediateToken = address(0)` versus `intermediateToken = USDC` produce byte-identical external calls and identical resulting state, so the field is dead weight that the NatSpec claims is used.

```
/// @param intermediateToken The intermediate token used for swapping
...
address intermediateToken;
```

Recommendation: Drop the field in this v2.0.0 ABI break, or, if it must be kept, restate its NatSpec as `/// @param intermediateToken Deprecated legacy ABI field; ignored by this facet`.

LI.FI: Fixed in [87c5733](#)

Researcher: Verified fix.

6.3.5 The MetaRouter branch accepts unauthenticated routing calldata emitting inaccurate LiFiTransferStarted event

Context: [SymbiosisFacet.sol#L227-L259](#)

Description: Setting `viaOnchainSwapV3 = false` skips `_verifyOnchainSwapV3Signature` and forwards every routing field verbatim into `metaRoute`.

```
if (_symbiosisData.viaOnchainSwapV3) {
    _startBridgeViaOnchainSwapV3(_bridgeData, _symbiosisData);
} else {
    _startBridgeViaMetaRouter(_bridgeData, _symbiosisData);
}
```

The facet never decodes `callData` and never compares it to `_bridgeData`; `validateBridgeData` only checks that receiver is nonzero, `minAmount` is nonzero and `destinationChainId != block.chainid`, none of which touch `callTo/callData`.

The deployed MetaRouter does not authenticate the recipient either, so the effective destination of the principal is fixed entirely by the caller-supplied payload while `LiFiTransferStarted` reports the declared `BridgeData`.

Recommendation: Accepting the risk is reasonable, in which case document it explicitly: state in the facet Nat-Spec and in `docs/` that on the MetaRouter path the emitted `BridgeData` is descriptive rather than enforced, so integrators and wallet clear-signing surfaces do not treat the displayed receiver and destination chain as on-chain guarantees.

Alternatively, extend the existing backend authentication to this branch in `_startBridgeViaMetaRouter`, before the `maxApproveERC20/metaRoute` call, and over the post-swap `_bridgeData.minAmount` for the swap endpoint:

```
bytes32 structHash = keccak256(
    abi.encode(
        METAROUTE_PAYLOAD_TYPEHASH,
        _bridgeData.transactionId,
        _bridgeData.minAmount,
        _bridgeData.sendingAssetId,
        _bridgeData.receiver,
        _bridgeData.destinationChainId,
        keccak256(_symbiosisData.firstSwapCalldata),
        keccak256(_symbiosisData.secondSwapCalldata),
        keccak256(abi.encode(_symbiosisData.approvedTokens)),
        _symbiosisData.firstDexRouter,
        _symbiosisData.secondDexRouter,
        _symbiosisData.callTo,
        keccak256(_symbiosisData.callData),
        _symbiosisData.deadline
    )
);
```

Reuse the existing `_domainSeparator()` and `backendSigner`, and apply the same single-use `transactionId` guard as the `OnchainSwapV3` branch, set before the external call.

LI.FI: Risk is documented in [84dbb29](#)

Researcher: Verified fix.